ChatGPT

# AI-Powered Exercise Form Validator: Methods, Models, and Implementation

## Overview

Building an AI-driven **training form validator** for exercises (e.g. squats or push-ups) involves using computer vision to analyze a user's workout video and provide feedback on their technique. The system must detect the user's body **pose**, evaluate if each repetition is performed correctly, and highlight mistakes with both images and text explanations. Key considerations include whether analysis runs on a **smartphone camera vs. specialized camera**, whether feedback is **real-time or after short processing**, deployment **on-device vs. cloud**, and whether to utilize existing **commercial APIs**. Below, we explore all viable implementation approaches as of **December 2025**, discuss the best models and workflows, and address challenges along with solutions.

## Pose Estimation: The Core Technology

At the heart of any form validator is **human pose estimation** – the detection of key body landmarks from images or video. Modern pose estimation models can track a person's skeleton (e.g. joints like shoulders, elbows, hips, knees) in real time from a normal RGB camera feed. Popular frameworks include **OpenPose**, **AlphaPose**, **MediaPipe/BlazePose**, and **YOLO-Pose**, among others [1] [2]. These systems output coordinates for keypoints (typically 17 to 33 landmarks) on the body for each video frame. For example, Google's **MediaPipe Pose** (used in ML Kit) provides 33 landmarks (covering full body) and even estimates their 3D orientation relative to the camera [3]. This technology does **not** require any special depth sensors or markers – a standard smartphone or webcam is sufficient, as demonstrated by projects running on just a webcam and Raspberry Pi [4]. High-quality or professional cameras can improve image clarity, but they are not strictly required for accurate pose detection under normal conditions.

**2D vs. 3D Pose:** Most implementations start with **2D pose estimation** (joint positions in the image). While 2D is often enough to evaluate form, it has limitations: depth ambiguities and certain angles (e.g. how far the knees travel forward) are hard to judge from a single view. Advanced approaches can reconstruct **3D poses** from one or multiple cameras to get true joint angles in space [5] [6]. Recent research shows monocular 3D pose models fine-tuned on fitness data (like the **InfiniteForm** dataset) can yield better accuracy for exercise poses [7] [8]. However, monocular 3D estimation is complex and may still struggle with occlusions. A simpler compromise is to use multiple 2D views: e.g. having the user film a squat from the side and front. Each angle offers unique insight – a **frontal view** shows left-right alignment (knees tracking over feet, symmetry of posture), while a **side view** better captures depth and spinal alignment [9] [10]. In practice, many systems just specify an optimal single view per exercise (for squats or push-ups, usually a side profile is recommended [10]) to ensure the camera sees the critical joints. **Special cameras** like the Microsoft Kinect (with depth sensor) can directly obtain 3D skeletons and were used in earlier solutions [11], but requiring such hardware is undesirable for consumer apps. Therefore, the best approach today is typically a well-tuned 2D pose model on regular smartphone video, possibly augmented with **AI-based 3D inference** if needed for complex assessments.

## Methods of Form Analysis

Once pose landmarks are obtained, there are two primary ways to assess exercise form: **rule-based geometric analysis** and **machine-learning classification**.

- **Rule-Based Angle & Position Checks:** This approach uses the coordinates of joints to calculate angles, distances, or alignments and then applies heuristic rules to decide if form is correct. It is relatively straightforward and interpretable. For example, a squat validator might compute the knee angle, hip angle, and torso lean at the bottom of the squat and check them against safe ranges [12]. Ideal joint angles for squats have been documented (e.g. hips around ~58° flexion, ankles ~81°, torso ~38° forward lean at the bottom in a proper deep squat) [13]. A push-up checker might measure the elbow angle to ensure the user bends sufficiently (e.g. <90° at bottom) and the body line (shoulder-hip-ankle) to ensure the back is straight. In fact, the Ultralytics YOLO-Pose toolkit has a built-in example that uses an elbow angle threshold of ~90° for "down" and ~145° for "up" to count push-up reps [14]. Similarly, one can check if during squats the thighs reach parallel to ground (hip-knee line angle) or if knees track properly. These rules can be coded by analyzing the triangle formed by relevant joints and using basic geometry. The system can then provide specific feedback like "Go deeper – your knees are not bending enough" or "Keep your back straight – avoid rounding." This method is effective for well-defined form criteria and has been used in many DIY projects and academic prototypes [4] [15]. It also enables highlighting the exact joints that are out of position (e.g. marking the knee if it's caving inward), a feature offered by some commercial SDKs [16].

- **Machine Learning Classification:** In more advanced implementations, one can train a machine learning model to classify or score the exercise form, rather than relying on fixed thresholds. Here, the input to the model could be the sequence of pose keypoints over time (or even the raw video frames), and the output could be a **"correct" vs "incorrect" posture classification**, or even multi-class labels indicating specific error types (e.g. "back bent" vs "knees too forward"). For instance, a 2023 study built a deep learning model that mimicked expert trainers' judgments on squat form [17] [18]. They extracted the 2D pose for each frame using OpenPose, selected the key joints relevant to squatting, and fed the trajectory of those keypoints into a custom neural network [19] [20]. Their model combined temporal **Conv1D layers and LSTM** blocks to analyze the motion over the entire squat repetition [21]. The result was a classifier that could label squat executions as correct or incorrect with ~85% accuracy on test data [22]. This outperformed a general-purpose video classifier (Google's AutoML Video) which only achieved ~69% on the same task [23]. The advantage of ML here is the ability to capture subtle, multi-joint patterns (for example, a slight hip shift or shaky descent) that are hard to reduce to simple rules. Over time, such models can be extended to output **specific feedback** if trained on categorized mistakes – essentially learning to recognize common faults (like "knees valgus" or "insufficient depth") by example. The downside is the need for a substantial dataset of example videos with expert labels to train the model, which for a startup might be a significant undertaking. Hybrid approaches are also used: one can apply basic angle filters to detect obvious issues and only use ML for borderline cases or overall scoring.

Regardless of method, the **workflow** typically looks like: *video frames -> pose keypoints -> analysis logic/model -> feedback output*. If doing rep-by-rep analysis, the system will segment the video into individual repetitions (often by detecting when a key angle crosses a threshold indicating the start/end of a rep) [24] [25]. It then evaluates each rep and can even give a numeric "form score." Some implementations smooth the data to avoid frame-to-frame noise – e.g. applying filters to the joint coordinates so that small jitter doesn't trigger

false errors [26]. In one real-time system, developers used a **Savitzky–Golay filter** on the angle signals to robustly identify the rep cycle without spurious counts [27]. The end result of analysis can be presented visually (overlaying the user's video with angles or colored lines for correct/incorrect alignment) and through text or voice feedback.

## Real-Time Feedback vs. Batch Analysis

A critical design choice is whether the form feedback is given **live (immediate)** or after the user finishes a set (with a slight delay for processing). Both are possible, and each has pros/cons:

- **Real-Time (Immediate) Feedback:** This means the system is analyzing the video on-the-fly as the user exercises, allowing it to give cues like "straighten your back" or "good rep" in that very moment. Achieving real-time feedback requires efficient processing – ideally running on the device (phone) itself to avoid network lag. The good news is that by 2025, mobile-friendly pose models are fast enough to do this. For example, Google's ML Kit Pose Detection API can run on a smartphone in real time, and researchers have demonstrated a mobile app that delivers *"immediate, actionable feedback to users during their workouts"* using this technology [28]. This mobile app (HALE) processed a squat video locally and even gave auditory/visual cues mid-set [29] [30]. Real-time feedback is great for helping users correct their form **during** the exercise, potentially preventing an improper rep or injury before it happens [28] [31]. However, it demands careful UI/UX design: users might not be looking at the screen while exercising, so feedback could be given via sound (a beep or voice prompt when form deviates) or minimalistic on-screen prompts they can glance at. Moreover, the computation must keep up with frame rate, which can be challenging for heavier models or when analyzing high-resolution video. Developers often choose a lightweight model (e.g. MediaPipe's BlazePose or MoveNet Lightning) for this scenario and may downsample the video feed for speed.

- **Batch (Post-Set) Analysis:** In this mode, the user records themselves doing a set of exercise, then stops and waits a short moment for the system to analyze the captured video. The app then returns feedback, including annotated frames showing mistakes and text explanations. This approach, used by services like **FormChecker**, provides near-instant results (FormChecker analyzes a 15-second clip in ~15 seconds) but not real-time coaching [32]. The trade-off here is a slight delay, but it allows using more powerful algorithms. For instance, the video could be uploaded to a **cloud server** where a heavyweight model (or even multiple models in pipeline) analyze it. With cloud GPU resources, one could run a very accurate pose model (or multiple viewpoint analysis) that wouldn't be feasible on a phone. The feedback might be more comprehensive as well – since the whole set is captured, the system can compute summary metrics (e.g. average depth, consistency of form across reps, etc.) and identify the **specific reps that were faulty**. Batch analysis is also simpler to implement initially: you don't have to optimize for millisecond timing, and the user can be asked to stay still for a moment after recording to receive feedback. In practice, many solutions aiming for **immediate insight** try to be as quick as possible even in batch mode. For example, FormChecker's selling point is giving feedback *"between exercise sets"* so that the user can correct things in the next set [33]. A delay of a few seconds to half a minute is usually acceptable in this scenario. If needed, a combination strategy can be adopted: basic cues in real-time (for critical form breaks) and a detailed report after the set is completed.

In summary, if **perfect form and safety** are the priority, real-time feedback is ideal as it can prevent bad reps. If **depth of analysis** and accuracy are priority, a short post-set analysis with heavier computation may be better. Many modern fitness apps try to incorporate elements of both for a good user experience.

## Deployment Architecture: On-Device vs. Cloud

Closely related to the real-time question is **where** the analysis runs – on the user's device or on cloud servers – or a hybrid of the two.

- **On Smartphone/Device:** Running the pose detection and analysis locally on a smartphone is feasible and offers significant benefits. It avoids any need to upload videos (protecting user privacy and working offline), and it minimizes latency (critical for real-time feedback). Mobile GPUs and neural accelerators (like Apple's Neural Engine or Qualcomm's DSP) can handle lightweight models. Google's **ML Kit Pose Detection** is a prime example: it's optimized to run in real-time on common phones and has been used in fitness apps for push-ups, squats, yoga, etc. [34]. In one case, a real-time squat/deadlift monitor app was built using ML Kit and confirmed to be accurate by testing with athletes [35] [36]. Similarly, open-source models like **MoveNet** (available through TensorFlow Lite) or **MediaPipe BlazePose** can achieve 30+ FPS on modern devices. The rule-based angle calculations are trivial for a phone's CPU once keypoints are obtained. Therefore, an entirely on-device pipeline is very possible for instant feedback. The challenge is ensuring compatibility across device models – older or low-end phones might struggle with real-time performance. To mitigate this, the app can adjust parameters (for instance, reduce frame rate or resolution if the device is slow). Another approach is to do **edge processing**, where some lightweight tasks run on device and heavier ones on a local server (e.g. a gym might have an on-premise server). But for a consumer app, sticking to on-device with a well-optimized model is usually the most scalable approach.

- **On Cloud Server:** Offloading analysis to a cloud server can be advantageous when either the model is too heavy for mobile or when the developer wants to avoid complicating the mobile app with ML code. The smartphone would simply capture video (possibly compress it) and send it to the server. The server could run a robust pose estimation (even multiple algorithms for cross-check) and perhaps a deep learning classifier for form. After analysis, it sends back results (like annotated frames and feedback text). This was the architecture used in the earlier **HALE squat app**: the Android client recorded a video, then sent it to a Python/TensorFlow server which did the pose and classification, returning the video with keypoints drawn and a correctness score [29] [30]. Cloud analysis opens up the possibility of using computationally intensive techniques – for example, you might run **OpenPose** (which is too slow for most phones) to get very accurate multi-person keypoints, or use ensembles of models for reliability. It also simplifies updating the model (you can improve the server algorithm without forcing users to update the app). However, the cloud approach has downsides: it requires a stable internet connection and introduces some delay (upload + processing time). There are **costs** to running servers with GPUs, especially if you plan to scale to many users – you may need to charge a subscription or use a usage-based pricing. Privacy is another consideration; users might be uncomfortable sending workout videos to the cloud. In many cases, developers compromise by doing as much as possible on-device and only using cloud for advanced analysis if the user opts in or if the device is incapable. Given that the question permits using **paid commercial services**, one could even leverage cloud APIs for parts of this – for example, using a cloud vision API to get poses (though Google's official APIs for pose are on-device ML Kit; OpenAI's vision or similar general APIs are not tailored for precise keypoints).

**Hybrid Solutions:** It's worth noting that some commercial SDKs blur the line between device and cloud. For instance, **QuickPose.ai** offers an SDK that integrates pose estimation into your app easily [37] – likely this runs on-device (they advertise an iOS/Android SDK) using a proprietary model. Others like **FormChecker** might essentially be cloud services where you upload clips to their platform for analysis [38]. When using any third-party service, one should evaluate latency, cost, and whether the service meets the specific needs (number of exercises supported, customizability of feedback). But if speed to market is a priority, these can jump-start development. For example, QuickPose specifically highlights capabilities like counting push-ups, checking form, and highlighting incorrect joints out-of-the-box [16] – in other words, many core features you need are pre-built in their SDK.

## Models and Tools (2025 State-of-the-Art)

With the above approaches in mind, let's enumerate some of the **best models and tools available** by late 2025 for each component:

- **Pose Estimation Models:** For 2D pose on a smartphone, **MediaPipe BlazePose** remains a top choice due to its speed and good accuracy on human body landmarks. It powers Google's ML Kit and infers 33 landmarks including hands and feet [39]. **MoveNet** (Google Brain) is another high-performance model; the "Thunder" version offers high accuracy and the "Lightning" version is optimized for mobile. **OpenPose** (by CMU) can detect full-body keypoints very reliably and even multiple people, but it's heavy – it might be used on a server if multi-person tracking or maximum precision is needed. **AlphaPose** and **HRNet** are other accurate models often used in research. Recently, **Ultralytics YOLO-Pose** models (e.g. YOLOv8-Pose, and now YOLOv11-Pose) have gained popularity. These combine object detection with pose estimation, allowing the model to focus on persons and output their keypoints in one go. Ultralytics provides a ready solution called **AIGym** that can feed a video and get pose + built-in workout metrics (they even have demos for push-up and pull-up counting) [2] [40]. For 3D pose, there are models like **VNect**, **PoseNet 3D**, and newer transformer-based approaches that attempt to predict 3D coordinates from a single view. Also, **Apple ARKit** (on iOS devices with LiDAR or dual cameras) offers an API for body tracking in 3D space – this could be leveraged on newer iPhones or iPads to get a 3D skeletal model without training your own. However, cross-platform apps tend to use the vendor-neutral approaches (TensorFlow, etc.).

- **Exercise Classification & Repetition Counting:** If you choose to incorporate an exercise type classifier (to automatically detect what exercise the user is doing), models like **YOLO** or MobileNet classifiers can be trained. The instructables project by M. Lenis, for example, trained a YOLOv11-based classifier to distinguish push-ups vs squats in real-time [41] [42]. This allowed the system to know which analysis logic to apply if multiple exercise types were possible. For repetition counting, simpler logic often suffices (detect peaks/troughs in joint angle over time), but one can also train a model to detect the cyclic movement. Some libraries like **OpenCV** or **MediaPipe Solutions** have templated code for basic exercise counting. In 2025, we also see **integrated platforms**: for instance, the **AI4Sports** or **GymML** type libraries that combine pose detection with domain logic. Ultralytics' AIGym (mentioned above) allows specifying which keypoints define the motion and angle thresholds, making counting easier [14].

- **Development Frameworks:** Python is commonly used for prototyping (with libraries like `opencv-python` for video and visualization, and PyTorch or TensorFlow for ML models). On mobile, if not using a provided SDK, one might use **TensorFlow Lite** to run a pose model. Google's ML Kit provides

a high-level interface (just call the pose detection function and it gives landmarks) in Android (Java/Kotlin) or iOS (Swift). **MediaPipe** also provides graphs that can be integrated in C++ or Android for pose tracking with filtering. For visualizing feedback, one may use OpenCV drawing functions or even game engines/AR libraries to overlay lines on the user's image.

- **Commercial Services:** We've mentioned **QuickPose.ai** SDK – it's a paid service but very relevant, aimed exactly at fitness apps with features like form feedback and joint highlighting [16]. Another one is **FormChecker** (AI fitness tool) which supports 30+ exercises by analyzing short clips and gives personalized feedback [38] [33]. These services likely have their own refined models (possibly an ensemble of pose estimation and posture classification) and save a lot of development time. If budget allows and the licensing fits your product, they can be integrated via API or SDK. On the other hand, using open-source components (MediaPipe, etc.) has no direct cost and gives you more control, at the expense of more engineering effort to refine the feedback logic.

## Workflow Example

Putting it all together, here's an example **workflow** for a squat/push-up validator application:

1. **User Recording:** The user places their smartphone such that their whole body is visible (for a squat, roughly chest-height camera, side view as recommended [30]; for a push-up, camera from side at ground level). The user hits record (or in live mode, just starts exercise in view). The app might give a 3-second countdown and guidance to get into starting position [29].

2. **Pose Detection:** As the user moves, each video frame is fed into the pose model. If on-device, this is done via an ML Kit or TFLite call. If on cloud, frames or the video clip are sent to the server. The output is a set of 2D (and possibly 3D) coordinates for key joints per frame.

3. **Data Processing:** The sequence of raw keypoints may be smoothed to remove noise [26]. The system identifies reps by looking for the characteristic motion pattern. For a push-up: detect when the person goes **down** (elbow angle decreasing below a threshold) and then back **up** (angle increasing past a threshold) [14]. For a squat: detect when the hip goes low and comes up. This can be done by monitoring joint angles (knee or hip angle) or vertical position of certain joints. Each full cycle increments a rep count.

4. **Form Evaluation:** For each rep (or continuously, in real-time mode), apply the chosen analysis method:

5. If rule-based: check angles at key points in the motion. For example, at the lowest point of the squat rep, measure if the thigh was at least parallel to ground (knee angle ~90°) or ideally below parallel (<80°) [12]. Check if the knees stayed roughly above the feet (which you can infer in front view by the horizontal distance between knee and ankle joints). Check torso angle – excessive forward lean or rounding might be flagged. In a push-up, at the lowest point check elbow angle (<90° means a deep push-up) and back alignment (angle between shoulder-hip-ankle should be ~180° – if hips sag or pike, that angle will deviate). These checks yield specific flags.

6. If ML model: run the posture classifier on the keypoint sequence for that rep or entire set. It might output a probability of "good form." If below a threshold, classify the rep as bad and possibly identify

the reason (some models might output a label, or one can infer reason by looking at which joint deviated most).

7. **Feedback Generation:** Based on the above evaluation, the system generates feedback:

8. **Visual:** This could be overlaying the user's video with skeleton lines and highlighting problematic joints or angles. For instance, drawing a red circle around a knee that went inward, or showing the angle values. Some apps create freeze-frames of the user at the lowest point of a bad rep and annotate it (e.g. an arrow indicating "back too rounded here"). Since the question asks for responding with images from the video highlighting errors, an implementation would extract those frames and programmatically mark them (perhaps with OpenCV drawing). This is exactly what QuickPose advertises – *"highlight joints that are in an incorrect position"* [16] .
9. **Text/Audio:** Alongside images, a text explanation is given. e.g. "Your squat depth was insufficient. Try to lower your hips more until your thighs are parallel to the floor." Or "Avoid flaring your elbows; keep them closer to your body during push-ups." The text is derived from whichever rule was broken or from the ML model's output. It's wise to keep feedback constructive and specific. If multiple issues occur, list them briefly. Some systems also give a numerical score or rating (e.g. 7/10 form score) to motivate improvement and track progress.

10. In real-time operation, feedback might be shorter phrases or sounds to not overload the user mid-exercise. For example, a quick "Too shallow!" audio prompt when a shallow squat is detected, or a rep *not* counted unless it meets criteria (gamifying it – some AI coaches only count a rep if done with proper form [2] ).

11. **Iterate or Log:** The user reviews the feedback. If it was real-time, they might adjust immediately on the next rep. If after-the-fact, they can use the insights in their next workout. The system can store data (with user permission) to show improvement over time – e.g. last session 3 out of 10 squats were flagged, this session only 1 out of 10, etc., giving a sense of progress.

Throughout this workflow, performance optimizations (downsampling frames, etc.) and accuracy checks (if pose detection fails for a frame, maybe skip or use last known pose) are applied to make it robust. Testing with different body types and camera setups is important to refine the system.

## Challenges and Solutions

Implementing this feature is very promising but comes with **several challenges**. Here we outline key challenges and how to address them:

- **Pose Estimation Errors:** Factors like poor lighting, busy background, or partial occlusion (e.g. arms blocking torso) can cause the pose model to mis-detect joints. This could lead to false feedback (flagging an error that wasn't actually there). **Solution:** Encourage proper setup – instruct the user to film in a well-lit area with the entire body visible. Many apps have a guide silhouette on screen to help the user position themselves correctly. Technically, use pose models with tracking capability (so they leverage temporal info to recover from momentary losses) and apply smoothing filters to ignore single-frame anomalies [26] . If a joint is occluded, often the model still guesses its position; some advanced models use human body kinematics to infer hidden joints. Testing the system on diverse scenarios will help tune it (for instance, if the user wears very baggy clothes, the system

might lose knee position – one might then warn in documentation to wear form-fitting clothes for best results).

- **Variability in User Body and Ability:** Users will have different limb lengths, flexibility, and may perform modified versions of exercises. One person's "deep squat" might be another's impossibility due to mobility issues. **Solution:** Personalization and settings. The app can have **difficulty modes** or user calibration. For example, the LearnOpenCV fitness trainer app offered *Beginner* vs *Pro* modes for squats, adjusting how strict the depth requirement is [43] [44]. Similarly, you might allow the user (or a trainer) to adjust the angle threshold for what counts as a good rep. Over time, the app could even learn the user's typical motion and gauge improvements relative to their past performance. Providing ranges (e.g. "you achieved 70° knee bend, aim for 80–90°") can be more constructive than a binary pass/fail. Also, if a user has an injury and shouldn't go deep, the system could be set to focus on form alignment instead of depth. Flexibility in the software logic prevents frustrating users whose anatomies or goals differ.

- **Multiple Camera Angles and 3D:** While multiple angles give more insight, combining them is non-trivial. If a user uploads two videos (front and side), the system would need to synchronize them (perhaps using timestamps or if recorded simultaneously). Merging two views for a coherent analysis (possibly reconstructing a pseudo-3D) is an advanced task. **Solution:** A simpler approach is to analyze each view separately for specific criteria. For instance, the side view video checks depth and back angle, the front view video (if provided) checks knee alignment and symmetry. The feedback can then aggregate those findings. Full 3D reconstruction from dual video is possible using algorithms like stereo triangulation if cameras were calibrated, but that's usually beyond consumer use. If a single camera is used, sticking to one optimal angle per exercise is recommended and should be clearly communicated ("Place your phone to your side such that it captures your full body profile"). Emerging single-view 3D pose models may ease this – if one can integrate a pre-trained 3D fitness pose model [45] [46], the system could get approximate depth info and perhaps compute joint angles in a more camera-angle-invariant way. This is cutting-edge, though, and might increase computational load.

- **Real-Time Performance Constraints:** Ensuring the app runs smoothly at interactive rates can be tough, especially on older devices. If the frame rate drops too low, feedback becomes laggy and rep counting might miss fast movements. **Solution:** Optimize at multiple levels. Use a lightweight pose model (there are ultra-light models that track fewer keypoints or trade some accuracy for speed). Use lower resolution video frames (you often don't need full HD for pose; camera input can be scaled down to e.g. 256x256 for the pose model, which is what BlazePose essentially does internally). Leverage device-specific accelerators: iOS's Vision framework or CoreML can run models on the neural engine; Android has NNAPI. Also manage the app's thread carefully – doing heavy ML on a background thread so the UI remains responsive. If a consistent 30 FPS is not attainable, consider providing feedback per rep rather than continuously per frame (this way you only need to correctly detect the key posture points of a rep, not every single moment). And, as mentioned, if needed fall back to analyzing after the fact when on a slow device.

- **Scalability and Cloud Costs:** If opting for cloud analysis, each video will consume bandwidth and server CPU/GPU time. For a large user base, this can become expensive. **Solution:** If using cloud, try to minimize the data sent (e.g. compress video, or even send just the keypoint data if the device can compute that). Some setups might do pose on-device and send only the coordinates to the server for

heavy analysis – coordinates are much lighter data. Also, employing a paywall or limits (like FormChecker's free tier allows 20 analyses [47]) can control costs. Using serverless or on-demand cloud functions that scale with usage can help economically. Over time, as device-side gets stronger, offloading more to the client is wise.

- **User Experience Challenges:** Getting users to actually use the feature correctly can be a challenge. They might find it cumbersome to record themselves, or they might not frame the camera right. **Solution:** Make the process as seamless as possible. Provide clear instructions and maybe an outline overlay to show where to stand. Possibly use voice guidance: "Step back a bit, you're too close to the camera" if the head or feet are cut off (this can be detected by whether keypoints are missing). Keep the feedback positive and helpful to encourage usage. Gamification (scores, rep counts, progress charts) can motivate users to engage regularly. Also, allow users to review past videos and feedback – this history can be valuable for them and for refining your algorithms (with user permission, collect anonymized data of mistakes to see common issues).

- **Accuracy vs. Strictness:** The system must balance being accurate with not being over-strict. If it flags almost every rep as "wrong", users will lose trust or get frustrated. But if it's too lenient, it defeats the purpose. **Solution:** Tune the thresholds and model outputs through extensive testing. It helps to involve fitness experts in the loop – have trainers test the app and see if they agree with the AI's feedback. The app could even offer a "strict mode" vs "casual mode" for feedback. As an example, in the squat study the app's "classification score threshold" for what counts as a correct squat was configurable by the user to adjust difficulty [48]. This kind of flexibility can accommodate different skill levels and preferences.

In addressing these challenges, leveraging the latest research and iterative testing is key. Notably, the field of AI fitness coaching has rapidly advanced due to the pandemic-driven demand for at-home training [49] [17], so by 2025 many of the above challenges have well-known mitigation strategies.

## Conclusion

In December 2025, developing a **training validator** feature is highly feasible thanks to advanced pose estimation and AI models. The **best implementation** likely combines a **fast pose model** (e.g. BlazePose via MediaPipe) with intelligent analysis logic that can be rule-based, ML-driven, or a hybrid. A typical workflow is to capture the user's exercise via a smartphone camera, extract pose landmarks, then either apply **joint angle heuristics** or an **expert-trained classifier** to identify form errors, and finally provide the user with **instant feedback** – possibly even in real time – including annotated images and corrective tips. The **workflow** can run entirely on-device for immediacy, or on a cloud backend for deeper analysis, depending on product needs. We have multiple **options** ranging from do-it-yourself open-source solutions to ready-made commercial APIs that can drastically speed up development [37].

The optimal choice of models and approach will depend on the specifics: for a simple use-case focusing only on squats and push-ups, a rule-based approach with a mobile pose model might suffice and yield responsiveness. For a more **comprehensive coach** that covers many exercises and nuanced feedback, investing in ML models (and perhaps cloud support) could pay off in providing a distinctive user experience. In all cases, attention must be paid to the **challenges** (pose errors, user variability, performance constraints, etc.), but as outlined, there are proven **solutions** to each. By incorporating these and staying

updated with the latest pose estimation improvements, one can build a robust training validator that helps users exercise safely and effectively – essentially offering a low-cost personal trainer powered by AI [50] [51].

**Sources:** The insights above are based on the latest research and implementations in AI fitness coaching. For example, Kim et al. (2023) demonstrated a smartphone-based squat coach using pose estimation and deep learning [17] [20]. Practical guides (e.g. by LearnOpenCV) show how MediaPipe can be used for real-time squat analysis with feedback logic [1] [9]. Commercial tools like FormChecker and QuickPose highlight the capabilities of current AI fitness services [38] [16]. Additionally, an instructables project by Lenis (2024) illustrates a full pipeline combining **MoveNet/MediaPipe** for pose and **YOLO** for exercise recognition to achieve real-time push-up and squat monitoring on low-cost hardware [52] [15]. These sources (and others cited throughout) collectively confirm that an AI-powered training validator is not only possible but already being realized with great success.

---


[1] [3] [9] [10] [39] [43] [44] AI Fitness Trainer - Build Using MediaPipe For Squat Analysis
https://learnopencv.com/ai-fitness-trainer-using-mediapipe/

[2] [14] [40] Workouts Monitoring using Ultralytics YOLO11 - Ultralytics YOLO Docs
https://docs.ultralytics.com/guides/workouts-monitoring/

[4] [15] [27] [41] [42] [52] Real-Time Push-Up & Squat Recognition With Form and Deepness Scoring : 6 Steps - Instructables
https://www.instructables.com/Real-Time-Push-Up-Squat-Recognition-With-Form-and-/

[5] [6] [7] [8] [45] [46] [50] web.stanford.edu
https://web.stanford.edu/class/cs231a/prev_projects_2022/qCS231A_Final_Paper.pdf

[11] [17] [18] [19] [20] [21] [22] [23] [26] [29] [30] [48] [49] An Artificial Intelligence Exercise Coaching Mobile App: Development and Randomized Controlled Trial to Verify Its Effectiveness in Posture Correction - PMC
https://pmc.ncbi.nlm.nih.gov/articles/PMC10523222/

[12] [13] [24] [25] [28] [31] [36] [51] Real-Time Biomechanical Squat and Deadlift Posture Analysis Using Google Machine Learning Kit
https://thesai.org/Downloads/Volume16No9/Paper_52-Real_Time_Biomechanical_Squat_and_Deadlift_Posture_Analysis.pdf

[16] [37] AI Push Up Counter - QuickPose.ai
https://quickpose.ai/exercise-library/fitness/ai-push-up-counter/

[32] [33] [38] [47] FormChecker Features, Pricing, and Alternatives | AI Tools
https://aitools.inc/tools/formchecker-ai

[34] ML Kit Pose Detection Makes Staying Active at Home Easier
https://developers.googleblog.com/ml-kit-pose-detection-makes-staying-active-at-home-easier/

[35] Real-Time Biomechanical Squat and Deadlift Posture Analysis Using Google Machine Learning Kit
https://thesai.org/Publications/ViewPaper?Volume=16&Issue=9&Code=IJACSA&SerialNo=52